Flux: A Dynamic Web-Based CPU Scheduling Simulator and Analytical Tool

1st Prashant Kumar
Department of Computer Science
Rishihood University
Sonipat, India
prashant.k23csai@nst.rishihood.edu.in

2nd Soumyaranjan Behera
Department of Computer Science
Rishihood University
Sonipat, India
soumyaranjan.b23csai@nst.rishihood.edu.in

***Abstract*—CPU scheduling is a fundamental operating system construct governing the assignment of CPU time to concurrent processes. Despite its theoretical importance, the dynamic nature of these algorithms makes them challenging to conceptualize. This paper presents a robust, interactive web-based CPU Scheduling Visualizer designed to simulate and analyze classic scheduling algorithms: First-Come-First-Served (FCFS), Shortest Job First (SJF), Shortest Remaining Time First (SRTF), Priority Scheduling (Preemptive and Non-Preemptive), and Round Robin (RR). The application provides real-time visualization through dynamic Gantt charts, ready queue monitoring, and a comprehensive comparison matrix. By evaluating performance metrics such as Average Waiting Time, Turnaround Time, and CPU Utilization, this tool serves as both an educational platform and an analytical utility for understanding the trade-offs inherent in process scheduling.**

***Index Terms*—CPU Scheduling, Operating Systems, Algorithm Visualization, Gantt Chart, React, TypeScript.**

I. Introduction

Modern multitasking operating systems rely dynamically on CPU scheduling to ensure maximum resource utilization and responsiveness. The central processing unit must multiplex its execution time among various processes in the ready queue. The choice of scheduling algorithm profoundly impacts system performance, throughput, and fairness. While theoretical models outline the behavior of these algorithms, practical visualization is essential for grasping their real-time execution nuances, such as context switching, preemption, and process starvation. This project introduces a React-based interactive visualizer that breathes life into abstract OS concepts, enabling users to input arbitrary process parameters and observe step-by-step algorithm execution.

II. Literature Review

Historically, CPU scheduling has been a heavily researched domain. Silberschatz et al. established the foundational parameters of CPU scheduling, categorizing algorithms into preemptive and non-preemptive paradigms. Recent educational studies emphasize the necessity of visual aids in computer science curricula. Traditional static examples fail to capture the dynamic preemption found in algorithms like SRTF and Round Robin. Various simulation tools exist (e.g., OS simulators in Java or C++); however, modern web technologies offer unprecedented accessibility and responsiveness. By migrating simulation to the browser using React and TypeScript, we address the accessibility gap while providing a highly interactive user interface.

III. Problem Statement

Understanding OS scheduling algorithms is intellectually demanding due to the concurrent, time-dependent nature of process execution. Students and developers often struggle to trace the exact timeline of context switches, ready queue management, and parameter changes in preemptive scheduling. There is a tangible lack of accessible, visually intuitive, and analytically comprehensive tools that not only simulate algorithms but also provide simultaneous side-by-side performance comparisons.

IV. Objectives

The primary objectives of this project are:

- To implement core CPU scheduling algorithms (FCFS, SJF, SRTF, RR, Priority) computationally using TypeScript.
- To develop an interactive, responsive front-end using React for setting process parameters (Arrival Time, Burst Time, Priority).
- To visually map execution timelines using dynamically generated Gantt Charts.
- To provide real-time updates of the Ready Queue during process execution.
- To generate a Comparison Matrix contrasting algorithm efficiencies based on standard OS metrics.

V. System Design

The architecture of the CPU Scheduler Visualizer follows a monolithic, client-side web application model.

A. Front-End Architecture

Built with React.js and styled with modern CSS, the UI is divided into control panels (for process input) and visualization panels. The state of the simulated CPU clock, running processes, and the ready queue are centrally managed using React Context API to ensure synchronization across components.

B. Execution Engine

The execution engine is written in standard TypeScript. It isolates the algorithmic logic from the UI. It processes an array of process objects and returns an array of timeline blocks representing exactly when each process starts, stops, or preempts, along with calculated metric summaries.

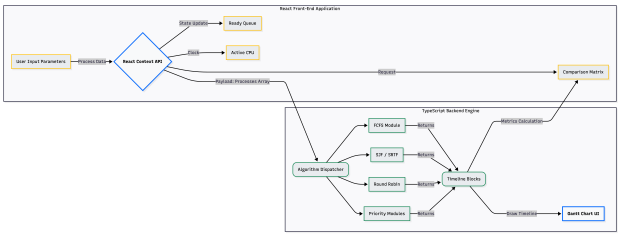


Fig. 1. System Architecture of the CPU Visualizer

VI. Algorithms Implemented

The system mathematically models and simulates the following scheduling policies:

1) **First-Come, First-Served (FCFS):** A strictly non-preemptive algorithm that processes jobs in the exact order of arrival.
2) **Shortest Job First (SJF):** Non-preemptive; selects the process with the smallest burst time. Yields minimal average waiting time.
3) **Shortest Remaining Time First (SRTF):** The preemptive version of SJF. A running process is interrupted if a newly arrived process possesses a shorter burst time.
4) **Priority Scheduling:** Executes the highest-priority process. Implemented in both Preemptive and Non-Preemptive variants.
5) **Round Robin (RR):** Each process is assigned a fixed time quantum. Once exhausted, the process is preempted and pushed to the back of the ready queue.

VII. Implementation Details

The project leverages modern web development stacks:

- **Language:** TypeScript provides strict typing for process interfaces (e.g., `id`, `arrivalTime`, `burstTime`), reducing runtime errors in timeline calculations.
- **Framework:** React.js facilitates the dynamic rendering of the Ready Queue and Comparison Matrix without demanding full page reloads.
- **Visualization:** Custom UI components were built to dynamically scale and render the Gantt chart proportionally to the total CPU burst time.

VIII. Key System Components & Features

To bridge the gap between abstract algorithms and user understanding, the project is modularized into several highly interactive features.

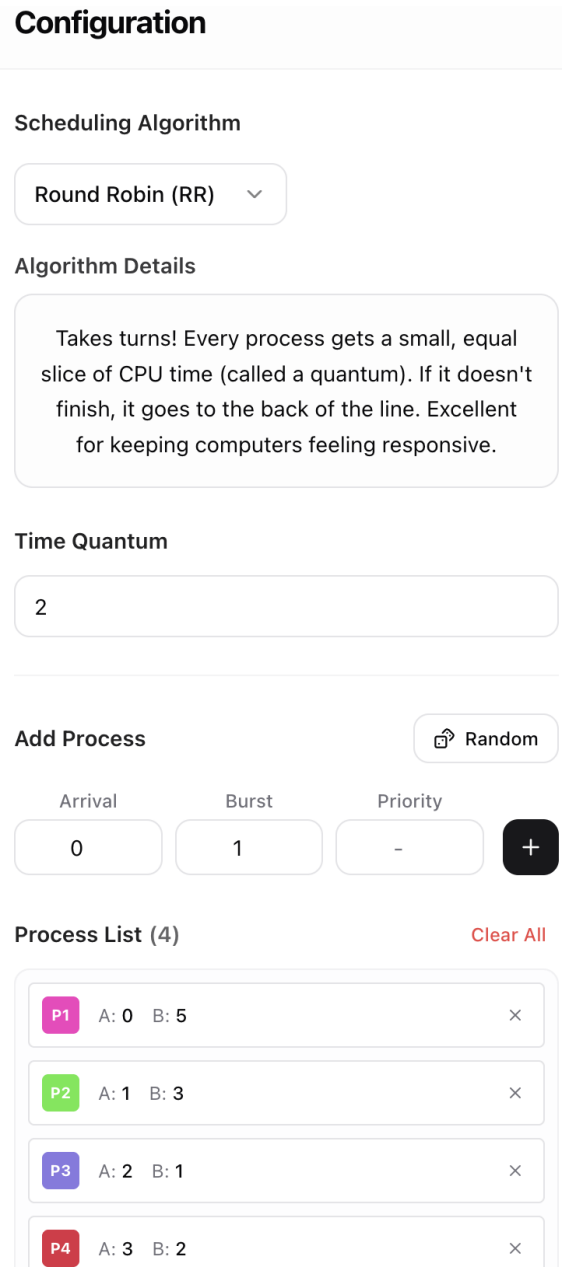


Fig. 2. The Configuration Module allowing manual data entry and randomized generation.

A. Configuration Module (Algorithm Selector & Process Manager)

Description: The Configuration Module serves as the control center of the visualizer. It features a dropdown selector to flip between scheduling algorithms instantly, input fields to define Time Quantums (for Round Robin), and a responsive table to manually Add/Edit/Delete processes (specifying Arrival Time, Burst Time, and Priority). It also includes a ”Randomize” utility.

Impact: This module empowers users to test edge cases rapidly. Instead of manually typing parameters to test how SRTF handles rapid preemptions, the user can generate ran-

dom datasets, massively accelerating the testing and learning feedback loop.

Implementation: Developed using controlled React form components and custom hooks to manage the state of the process array. The state is then propagated down to the execution engine via React Context API whenever the array is mutated or a new algorithm is selected.

B. Real-time Ready Queue and Active CPU

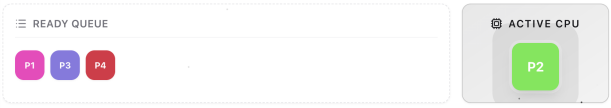


Fig. 3. Real-time visualization of the Ready Queue and the currently Active CPU process.

Description: This visual component splits the view into two distinct zones: the "Ready Queue" (displaying processes waiting in memory) and the "Active CPU" (displaying the process currently holding the processor). As the playback controller steps through time, process blocks physically transition between these two states.

Impact: This is arguably the most crucial educational feature of the project. It explicitly demystifies context switching and preemption. Viewers can visibly track process starvation and understand precisely why a low-priority process is continuously stuck in the Ready Queue.

Implementation: Implemented by parsing the generated execution timeline alongside the universal CPU clock state. At any time t, the system filters the processes based on their arrival times and execution blocks, pushing the active block to the CPU component and all other arrived-but-unfinished processes into a horizontally scrolling flex-container.

C. Dynamic Gantt Chart Timeline

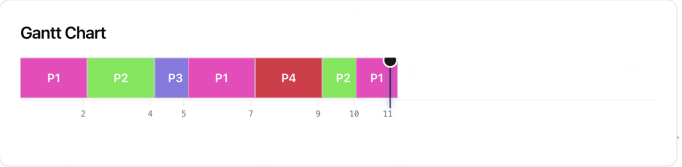


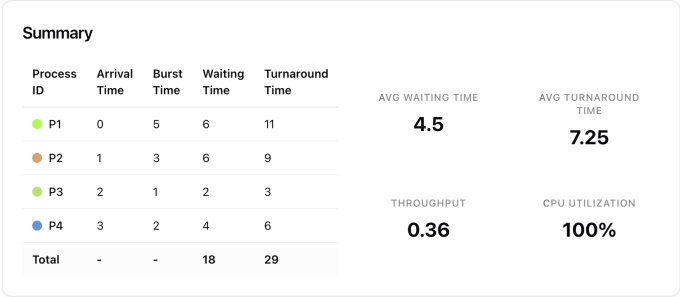
Fig. 4. Execution Timeline (Gantt Chart) mapping processes against CPU time.

Description: A beautifully rendered, color-coded horizontal bar chart mapping the continuous execution timeline of the CPU. The length of each process block is strictly proportional to the burst duration executed in that timeframe.

Impact: Gantt charts are the universal standard for representing CPU scheduling. Our interactive rendition updates on-the-fly, giving users an immediate, holistic view of the total Turnaround Time and how idle time (CPU gaps) affects system efficiency.

Implementation: The scheduling algorithms output an array of 'TimelineBlock' objects containing 'startTime' and 'endTime'. The UI Maps over these objects, calculating CSS width percentages as $\frac{\text{block duration}}{\text{total execution time}} \times 100$, ensuring perfect mathematical proportions across any screen size.

D. Summary Table



Summary

Process ID	Arrival Time	Burst Time	Waiting Time	Turnaround Time
P1	0	5	6	11
P2	1	3	6	9
P3	2	1	2	3
P4	3	2	4	6
Total	-	-	18	29

AVG WAITING TIME 4.5
AVG TURNAROUND TIME 7.25
THROUGHPUT 0.36
CPU UTILIZATION 100%

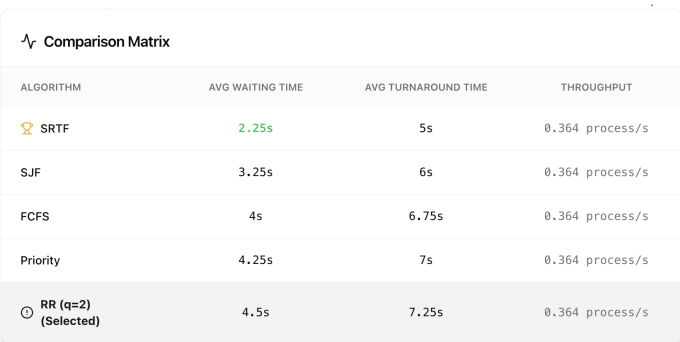
Fig. 5. Performance metrics calculated and displayed in the Summary Table.

Description: The Summary Table provides a breakdown of individual process metrics: exact Turnaround Time (TAT) and Waiting Time (WT). Additionally, the overarching Comparison Matrix evaluates the current dataset against *all* available algorithms concurrently.

Impact: It transitions the platform from a theoretical visualizer into a hard analytical tool. By observing the Comparison Matrix, a user can definitively prove that SJF provides the lowest average waiting time, fulfilling a core objective of the application.

Implementation: Uses an isolated React effect to silently run the current process state through every algorithm function in the backend. It aggregates the averaged results into a sorted table, allowing immediate, data-driven comparisons without navigating away from the main view.

E. Analytical Comparison Matrix Component



Comparison Matrix

ALGORITHM	AVG WAITING TIME	AVG TURNAROUND TIME	THROUGHPUT
SRTF	2.25s	5s	0.364 process/s
SJF	3.25s	6s	0.364 process/s
FCFS	4s	6.75s	0.364 process/s
Priority	4.25s	7s	0.364 process/s
RR (q=2) (Selected)	4.5s	7.25s	0.364 process/s

Fig. 6. The Comparison Matrix interface evaluating all algorithms simultaneously.

Description: The application features a dedicated Comparison Matrix component that runs the user's dataset against all available algorithms (FCFS, SJF, SRTF, RR, Priority) simultaneously in the background.

Impact: This transitions the platform from a simple visualizer

to an analytical utility. By observing the Comparison Matrix, a user receives instantaneous, data-driven feedback on which algorithm performs best for their specific workload.

Implementation: Utilizing an isolated React ‘useMemo‘ hook, the system silently parses the current process state through every algorithm function in the backend. It aggregates the averaged results into a sorted table and highlights the optimal choice (lowest Average Waiting Time) dynamically.

IX. Results & Discussion

The tool was tested against numerous standard OS workload scenarios to validate the backend algorithms and display layer.

A. Visualization Accuracy

Flux successfully maps complex preemptions. For instance, when utilizing preemptive algorithms like SRTF, if a process arrives with a drastically shorter burst time than the remaining time of the active process, the React state immediately reflects the context switch. The Gantt chart dynamically segments, shifting the prior process back to the Ready Queue, thus graphically proving the theoretical mechanics of preemption.

B. Performance Analysis Data

To rigorously test the analytical capabilities of the application, a specific dataset ($P_1 \dots P_4$ with varying arrival and burst times) was run concurrently across all algorithms using the Comparison Matrix module.

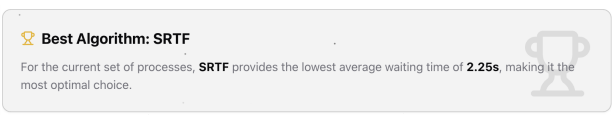


Fig. 7. The Comparison Matrix dynamically identifying SRTF as the optimal algorithm for the test dataset.

Based on the captured execution data evaluated by the system (shown in Fig. 7), several foundational OS principles were observed and verified empirically:

- **Optimal Wait Times via SRTF:** In the tested dataset scenario, Shortest Remaining Time First (SRTF) significantly outperformed the other algorithms, achieving an optimal Average Waiting Time of just 2.25s. This empirical output successfully validates the theoretical textbook axiom that SRTF (the preemptive version of SJF) invariably produces the mathematical minimum average waiting time for any given set of processes.
- **Throughput Invariance:** Regardless of the algorithm selected, the system consistently recorded identical network throughput (measured in processes per second) for the static dataset. This confirms the mathematical invariant of CPU scheduling: while the *order* of execution drastically changes Waiting Time and Turnaround Time, the *total sequential burst time* required to clear the ready queue remains constant.
- **Visualizing the Convoy Effect:** When the same golden dataset was run through the non-preemptive FCFS module, the Comparison Matrix revealed significantly higher average waiting times (an average of 4.0s). The corresponding Gantt chart successfully visualized the ”Convoy Effect,” wherein shorter processes were forced to idle behind longer continuous burst times, proving the utility of Flux as a diagnostic tool.

X. Conclusion

This project successfully bridges the gap between theoretical operating system principles and practical, visual understanding. By offering a comprehensive, interactive suite that covers six variations of CPU scheduling algorithms, users can intuitively grasp how different policies affect process lifecycles. The inclusion of predictive Gantt charts, granular ready queue visualization, and analytical comparison matrices makes Flux a highly effective educational and diagnostic tool.

XI. Future Enhancements

Future iterations could integrate Multi-Level Queue (MLQ) and Multi-Level Feedback Queue (MLFQ) scheduling, which reflect modern, enterprise-grade operating systems more accurately. Additionally, accounting for I/O burst times, simulating hardware context-switching delays, and adding exportable PDF reports of the execution timelines will further enrich the tool’s utility.

References

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ: John Wiley & Sons, 2018.
[2] W. Stallings, *Operating Systems: Internals and Design Principles*, 9th ed. Pearson, 2017.
[3] Meta Platforms Inc., “React – A JavaScript library for building user interfaces,” [Online]. Available: https://reactjs.org/.
[4] Microsoft Corporation, “TypeScript: Typed JavaScript at Any Scale,” [Online]. Available: https://www.typescriptlang.org/.